

Customer Support RAG-Powered Intelligent Chatbot

Project 5

GIZ4_AIS2_S3

Names
Khaled Mahmoud (Team Leader)
Khaled Mohammed
Farah Awadalla
Mohammed Basel
Zeyad Mohammed
Rana Alaa

20/04/2026

GitHub Link: <https://github.com/KhaledEldodor282/RAG-Customer-Support-Chatbot>

1. Project Planning & Management

1.1 Project Proposal

Overview

This project aims to develop a **production-ready customer support chatbot** powered by **Retrieval-Augmented Generation (RAG)**. The system will handle real user queries by retrieving relevant information from a knowledge base (support tickets, FAQs, and documentation) and generating accurate, context-aware responses.

The chatbot will integrate with backend systems through APIs and leverage modern NLP techniques, transformer models, and vector search.

Objectives:

- Build an intelligent chatbot for customer support
- Implement a RAG pipeline for accurate response generation
- Reduce response time and manual workload
- Integrate the system with backend services via APIs
- Ensure scalability and production readiness

Scope:

The project includes:

- Data collection (tickets, FAQs, documents)
- Text preprocessing and embeddings generation
- RAG model development
- API development and deployment (Azure)
- Monitoring and performance tracking

1.2 Project Plan (Timeline)

1.3 Milestones & Deliverables

- **Milestone 1:** Processed dataset + EDA report
- **Milestone 2:** Trained RAG pipeline + evaluation
- **Milestone 3:** Deployed chatbot API
- **Milestone 4:** Monitoring dashboard + retraining pipeline
- **Milestone 5:** Final report + demo

1.4 Task Assignment & Roles

- **Member 1:** Data collection, preprocessing, EDA
- **Member 2:** Embeddings + vector database
- **Member 3:** RAG model configuration
- **Member 4:** Backend API & Azure deployment
- **Member 5:** Documentation, testing, KPIs

1.5 Risk Assessment & Mitigation

Risk	Impact	Mitigation
Poor data quality	Weak responses	Use cleaning + augmentation
Limited dataset	Low relevance	Combine multiple sources
Deployment issues	System downtime	Use Azure managed services
High latency	Poor UX	Optimize retrieval & caching

1.6 KPIs (Key Performance Indicators)

- Response Accuracy (BLEU / ROUGE)
- Response Relevance Score
- API Response Time < 2 seconds
- System Uptime ≥ 95%
- User Satisfaction Rate

2. Literature Review

2.1 Introduction

Customer support systems are essential for handling user queries efficiently. Traditional systems rely heavily on human agents, which often leads to delays, inconsistency, and scalability limitations. While early automation attempts improved efficiency, they still lacked deep contextual understanding of user intent. Recent advancements in Natural Language Processing (NLP) and deep learning have significantly improved the ability of systems to process and respond to human language, although challenges such as hallucination and domain adaptation remain (Jurafsky & Martin, 2023).

2.2 NLP in Customer Support

Natural Language Processing (NLP) techniques are widely used to process and structure unorganized text data in support tickets. Traditional approaches such as rule-based systems and keyword matching were effective in controlled environments but failed to generalize across diverse user inputs.

Although classical machine learning models improved performance, they still rely heavily on manual feature engineering, which limits scalability and adaptability to new domains. This indicates that earlier NLP systems were constrained by their dependence on handcrafted rules and shallow representations of text (Bird, Klein, & Loper, 2009).

2.3 Transformer Models

Transformer-based architectures such as BERT and GPT have significantly advanced the field of NLP by introducing attention mechanisms that capture contextual relationships within text. Compared to RNNs and LSTMs, transformers eliminate sequential processing limitations, enabling better long-range dependency modeling.

However, despite their strong performance, transformer models are computationally expensive and often require large datasets for effective fine-tuning. Additionally, they may generate plausible but factually incorrect responses when used in open-ended generation tasks, which limits their reliability in production-level support systems (Vaswani et al., 2017; Devlin et al., 2019).

2.4 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) improves upon traditional language models by integrating external knowledge retrieval with generative capabilities. Instead of relying solely on learned parameters, RAG retrieves relevant documents from a knowledge base, which helps ground generated responses in factual information.

This approach significantly reduces hallucination compared to standalone generative models. However, RAG systems introduce additional complexity in terms of retrieval quality and latency. If the retrieval component fails to fetch relevant documents, the overall system performance degrades, indicating a dependency between retrieval accuracy and generation quality (Lewis et al., 2020).

2.5 Vector Search and Embeddings

Vector databases such as FAISS and Azure Cognitive Search enable semantic search by converting text into high-dimensional embeddings. This allows systems to retrieve conceptually similar documents rather than relying on exact keyword matches.

Although vector search improves retrieval accuracy, it introduces challenges related to embedding quality and indexing efficiency. Poorly trained embeddings can lead to irrelevant retrieval results, which directly impacts downstream generation performance. Therefore, embedding quality becomes a critical bottleneck in RAG-based systems (Johnson, Douze, & Jégou, 2019).

2.6 Comparative Analysis of Approaches

Traditional chatbot systems are simple and computationally efficient but lack flexibility and contextual understanding. Transformer-based models significantly improve language understanding but suffer from hallucination and high computational cost.

RAG systems attempt to bridge this gap by combining retrieval with generation, resulting in more factually grounded outputs. However, this hybrid approach introduces system complexity and dependency on external knowledge sources. Therefore, while RAG improves response quality, it also requires careful optimization of both retrieval and generation components to achieve stable performance.

2.7 Research Gap

Despite advancements in NLP and generative AI, most existing systems focus either on standalone classification models or pure generative chatbots. Few systems integrate retrieval, generation, deployment, and monitoring into a single production-ready pipeline.

Furthermore, limited research addresses the operational challenges of deploying RAG systems at scale, such as latency optimization, vector database tuning, and continuous monitoring. This highlights a gap between research prototypes and real-world production systems.

This project addresses this gap by developing a full-stack RAG-powered chatbot integrated with backend APIs, vector search, and MLOps monitoring tools.

2.8 Conclusion

The literature shows a clear evolution from rule-based systems to transformer models and finally to retrieval-augmented generation systems. While each stage improves performance, none fully solve issues of scalability, factual accuracy, and production deployment individually. Therefore, integrating these techniques into a unified system presents a promising direction for real-world customer support automation.

2.9 References

- Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL*.
<https://doi.org/10.18653/v1/N19-1423>
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*. <https://doi.org/10.1109/TBDATA.2019.2921572>
- Jurafsky, D., & Martin, J. H. (2023). *Speech and language processing*. Stanford Draft.
- Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*. <https://arxiv.org/abs/2005.11401>
- Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*. <https://arxiv.org/abs/1706.03762>

3. Requirements Gathering

3.1 Stakeholder Analysis

The system involves multiple stakeholders with distinct needs, each influencing system design requirements.

Stakeholder	Needs	System Implication
Customers	Fast, accurate responses to queries	Requires low-latency RAG pipeline
Support Agents	Reduced manual workload	Requires automation of ticket understanding
System Administrators	Monitoring and control of system performance	Requires dashboards and logging
Developers	Maintainable and scalable architecture	Requires modular API and pipeline design

From a systems perspective, the success of the chatbot depends on balancing user experience requirements (speed and accuracy) with technical constraints such as latency, scalability, and retrieval quality.

3.2 User Stories

User stories define how different users interact with the system and highlight functional expectations.

- As a **customer**, I want to receive accurate answers instantly so that I do not need to wait for human support.
- As a **support agent**, I want the system to automatically understand and classify incoming queries so that my workload is reduced.
- As an **administrator**, I want to monitor system performance and errors so that I can ensure reliability and uptime.

These user stories emphasize the need for both intelligent understanding (NLP/RAG) and operational reliability (monitoring and APIs).

3.3 Use Cases

◇ Use Case 1: Customer Query Handling

1. User submits a query through chatbot interface
2. System preprocesses and converts query into embeddings
3. Vector database retrieves relevant documents
4. RAG model generates a context-aware response
5. Response is returned to the user

Critical Note:

Unlike traditional chatbots that rely on predefined responses, this system dynamically retrieves knowledge, improving accuracy and adaptability.

◇ Use Case 2: Knowledge Retrieval

1. System receives query embedding
2. Similarity search is performed in vector database
3. Top-k relevant documents are retrieved
4. Retrieved context is passed to generation model

This separation between retrieval and generation improves factual grounding but introduces dependency on retrieval quality, making embedding design a critical component.

3.4 Functional Requirements

The system must implement the following core functionalities:

- Accept natural language user queries through API endpoints
- Preprocess text (tokenization, cleaning, normalization)
- Convert queries into semantic embeddings
- Perform similarity search using vector database
- Retrieve relevant knowledge base documents
- Generate responses using a RAG pipeline
- Return structured responses via REST API
- Log interactions for monitoring and evaluation

Critical Insight:

Unlike traditional systems, functionality is not limited to classification; instead, the system must integrate retrieval and generation into a unified pipeline, requiring careful orchestration between components.

3.5 Non-Functional Requirements

◇ Performance

The system must maintain low latency (target < 2 seconds per query). This is critical because retrieval and generation are both computationally expensive operations.

◇ Scalability

The architecture must support increasing numbers of concurrent users without degradation in performance, requiring scalable vector storage and API design.

◇ Reliability

The system must maintain consistent performance under load. Failures in retrieval or model inference should be handled gracefully.

◇ Security

User queries and retrieved data must be securely processed. API endpoints should be protected using authentication mechanisms such as API keys or Azure AD.

◇ Maintainability

The system must follow modular design principles to allow independent updates to retrieval, embedding, and generation components.

3.6 System Constraints

- Limited availability of high-quality labeled support ticket data
- High computational cost of transformer-based models
- Dependency on vector database performance
- Cloud resource limitations during deployment

These constraints directly influence design decisions, particularly in model selection and system architecture optimization.

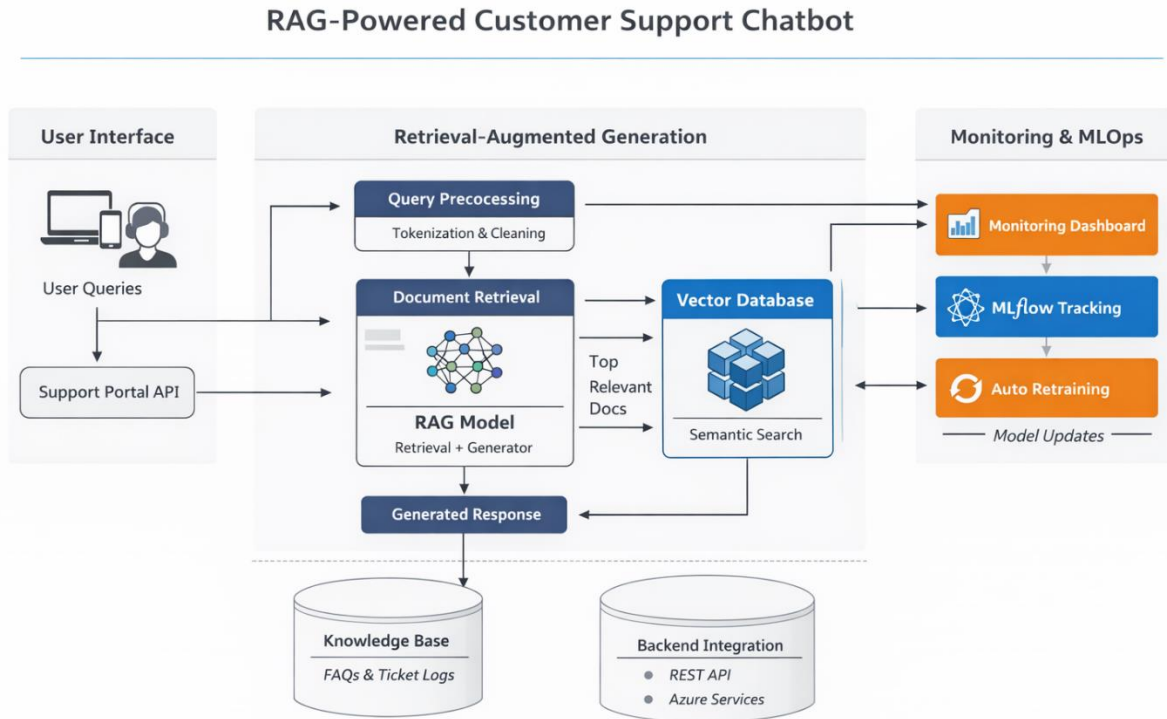
3.7 Assumptions

- Users will submit text-based queries only
- Knowledge base (tickets + FAQs) is sufficiently representative of real queries
- Cloud deployment environment (Azure) is available and stable

3.8 Conclusion

The requirements analysis highlights that the system is not simply a chatbot, but a **full retrieval-augmented intelligence pipeline**. The integration of NLP, vector search, and generative models introduces both functional capabilities and architectural complexity. Therefore, careful design of retrieval quality, latency optimization, and modular system architecture is essential for production readiness.

4. Architecture Design



4. System Analysis & Design

This section presents the analytical and design foundation of the Customer Support RAG-Powered Intelligent Chatbot. It defines the problem being addressed, the goals the system must achieve, the actors and use cases that shape its functional scope, the requirements that constrain its implementation, and the architectural, data, and behavioral models used to design the solution. Together, these artifacts translate the project's business objectives into a blueprint that guides development, database design, and deployment.

4.1 Problem Statement & Objectives

4.1.1 Problem Statement

Modern support organizations receive a high volume of repetitive customer queries through channels such as social media, email, and live chat. Traditional support workflows rely heavily on human agents to search for scattered documentation FAQs, product manuals, and historical tickets — before formulating a reply. This manual retrieval process is slow, inconsistent, and does not scale well with growing customer bases, leading to long response times, uneven answer quality, and agent burnout. There is a need for an intelligent system that can automatically understand a customer's question, retrieve the most relevant knowledge from internal documentation, and generate an accurate, context-aware response in real time, while still allowing a human agent to intervene when the system's confidence is low.

4.1.2 Objectives

The Customer Support RAG-Powered Intelligent Chatbot is designed to meet the following objectives:

- Automate first-line customer support responses using Retrieval-Augmented Generation (RAG) so that answers are grounded in verified internal knowledge rather than the language model's unguided output.
- Reduce average response time by removing manual document search from the support workflow.
- Improve answer relevance and accuracy through semantic retrieval over a vector database of embedded knowledge documents.
- Integrate seamlessly with existing business backend systems (ticketing, authentication, and reporting) through secure REST APIs.
- Enable continuous monitoring of system performance and automated retraining / embedding refresh so accuracy improves over time as new support data becomes available.
- Provide a graceful escalation path to human agents for queries the system cannot answer confidently.

4.2 Use Case Diagram & Descriptions

The use case diagram below identifies the primary actors that interact with the system and the functionality each actor can invoke. Four actors are recognized: the Customer, who submits queries and

receives responses; the Support Agent, who handles escalated conversations; the System Administrator, who manages the knowledge base and oversees system health; and the external LLM Provider, a supporting system actor that supplies the underlying language-generation capability.

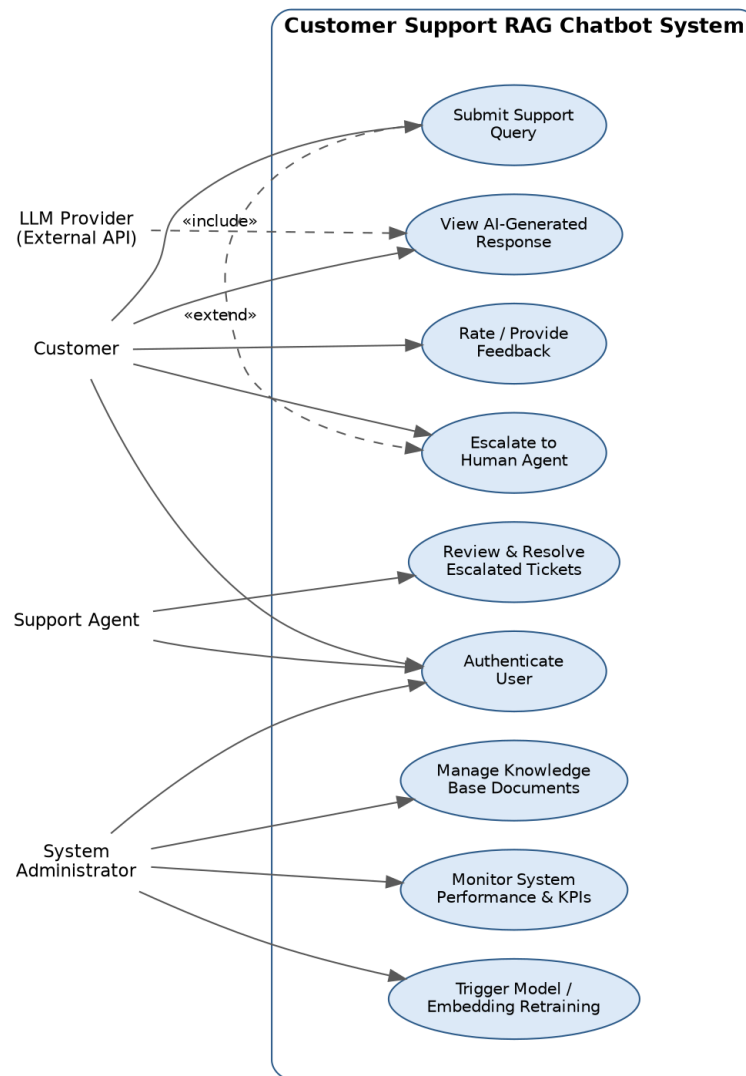


Figure 4.1 — Use Case Diagram of the Customer Support RAG Chatbot

4.2.1 Use Case Descriptions

Use Case	Primary Actor	Description
Submit Support Query	Customer	Customer types a natural-language question into the chat widget; the query is sent to the RAG pipeline for processing.
View AI-Generated Response	Customer	System displays the generated, context-grounded answer along with cited source documents.
Rate / Provide Feedback	Customer	Customer rates the usefulness of a response, which is stored for quality monitoring and retraining.
Escalate to Human Agent	Customer	Customer requests (or the system automatically triggers) hand-off of the conversation to a live agent.
Review & Resolve Escalated Tickets	Support Agent	Agent views escalated conversation history and knowledge context, then resolves the customer issue.
Manage Knowledge Base Documents	Administrator	Administrator uploads, edits, or removes FAQs, manuals, and knowledge documents that feed the vector store.
Monitor System Performance & KPIs	Administrator	Administrator views dashboards for latency, retrieval accuracy, escalation rate, and customer satisfaction.
Trigger Model / Embedding Retraining	Administrator	Administrator initiates or schedules re-embedding of updated documents and model fine-tuning via MLflow pipelines.
Authenticate User	All Actors	Shared use case invoked before any protected action; verifies identity via Azure AD or API key.

4.3 Functional & Non-Functional Requirements

4.3.1 Functional Requirements

ID	Requirement
FR-1	The system shall accept free-text customer queries through a REST API and web/mobile chat interface.
FR-2	The system shall preprocess incoming queries (cleaning, tokenization, normalization) before retrieval.
FR-3	The system shall convert queries into vector embeddings and perform similarity search against the vector store.
FR-4	The system shall retrieve the top-K most relevant knowledge chunks and pass them as context to the language model.
FR-5	The system shall generate a natural-language response grounded in the retrieved context and return it with source citations.
FR-6	The system shall allow customers to rate responses and submit free-text feedback.

FR-7	The system shall automatically escalate a conversation to a human agent when the retrieval confidence score falls below a configurable threshold, or when explicitly requested.
FR-8	The system shall allow administrators to add, update, or remove knowledge base documents, triggering re-embedding of affected content.
FR-9	The system shall log every query, response, latency, and evaluation metric (BLEU, ROUGE, relevance score) to MLflow for tracking.
FR-10	The system shall support scheduled or manually triggered retraining / embedding-refresh pipelines.
FR-11	The system shall authenticate and authorize all API requests using Azure AD or API keys.

4.3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	The system shall return a generated response within 3 seconds for 95% of queries under normal load.
NFR-2	Scalability	The system shall scale horizontally on Azure App Service / AKS to handle peak concurrent chat sessions without degradation.
NFR-3	Availability	The system shall maintain 99.5% uptime, excluding scheduled maintenance windows.
NFR-4	Security	All data in transit shall be encrypted via TLS 1.2+, and access to administrative functions shall require role-based authentication.
NFR-5	Reliability	The system shall gracefully degrade to human escalation if the LLM provider or vector store is unavailable.
NFR-6	Maintainability	The RAG pipeline components (retriever, generator, post-processor) shall be independently deployable and testable.
NFR-7	Usability	The chat interface shall be accessible (WCAG 2.1 AA) and usable on both desktop and mobile devices.
NFR-8	Auditability	All conversations, model versions, and retraining events shall be logged and traceable via MLflow run IDs.

4.4 Software Architecture

The system follows a layered, service-oriented architecture that separates concerns between the client interface, request handling, core RAG processing, persistent storage, and MLOps monitoring. Each layer communicates through well-defined REST interfaces, which allows individual components — such as the retriever or the generator — to be scaled, replaced, or upgraded independently. This design borrows principles from microservices architecture for the RAG application layer while keeping the overall deployment footprint manageable on Azure App Service / Azure Machine Learning endpoints.

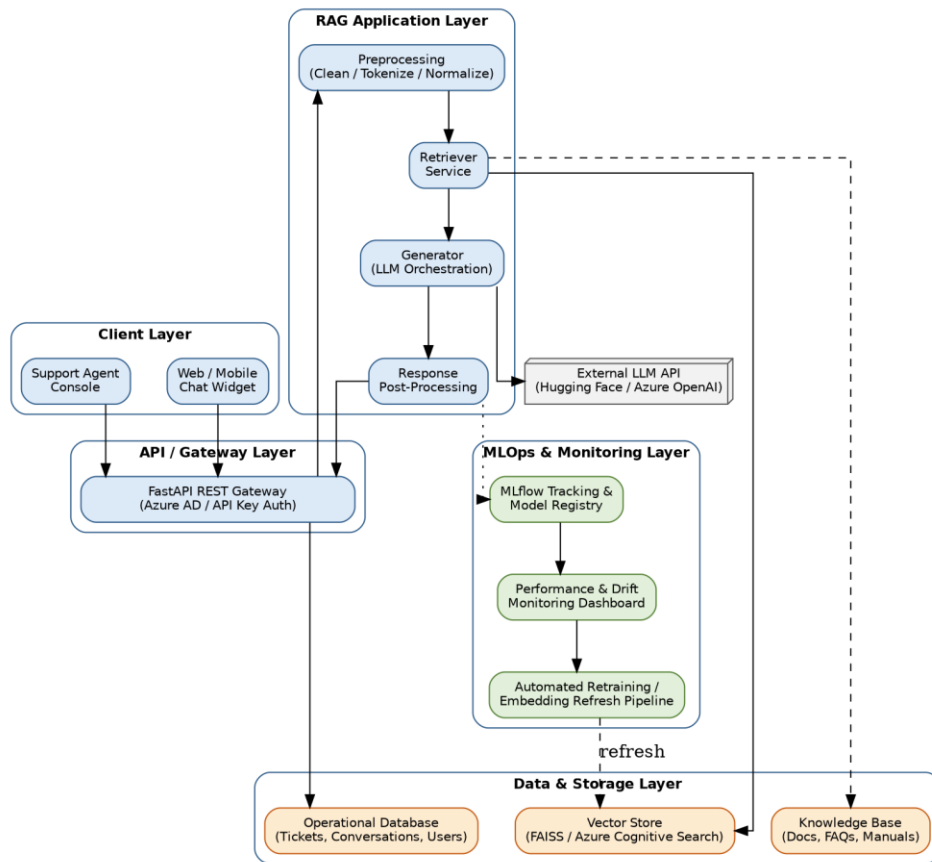


Figure 4.2 — Software / Component Architecture of the RAG Chatbot System

- **Client Layer:** Web/mobile chat widget and support-agent console through which users and agents interact with the system.
- **API / Gateway Layer:** A FastAPI-based REST gateway that authenticates requests (Azure AD / API keys) and routes them to backend services.
- **RAG Application Layer:** Preprocessing, Retriever, Generator, and Response Post-Processing components that together implement the retrieval-augmented generation workflow.
- **Data & Storage Layer:** The vector store (FAISS / Azure Cognitive Search) for embeddings, an operational database for tickets and conversations, and the raw knowledge base document store.
- **MLOps & Monitoring Layer:** MLflow experiment tracking, a performance/drift monitoring dashboard, and an automated retraining and embedding-refresh pipeline.

4.5 Database Design & Data Modeling

4.5.1 Entity-Relationship Diagram

The ER diagram below models the core persistent entities of the system: users, conversations, messages, escalated tickets, support agents, knowledge documents and their chunks/embeddings, customer feedback, and model version metadata used for retraining traceability.

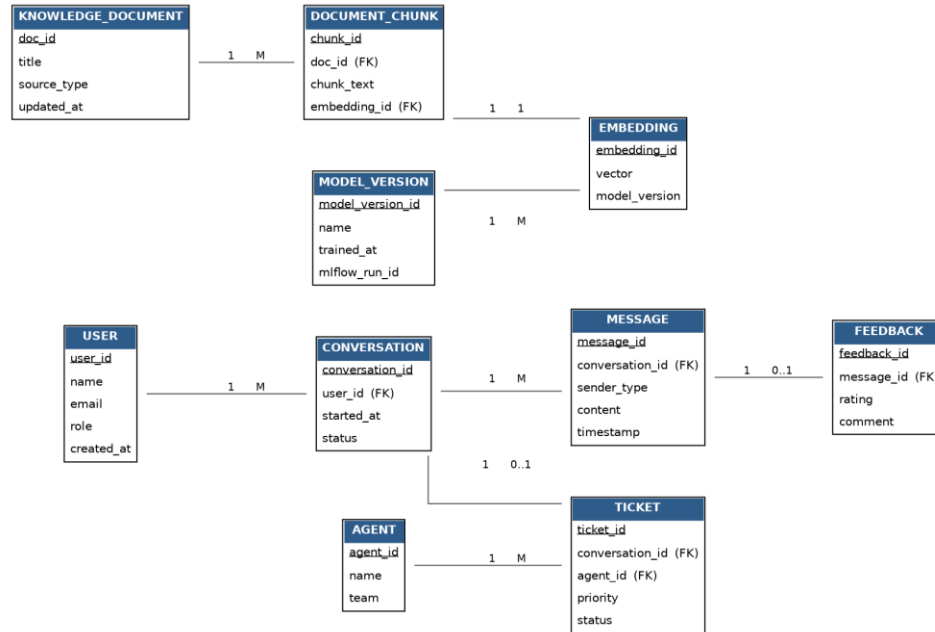


Figure 4.3 — Entity-Relationship Diagram (ERD)

4.5.2 Logical & Physical Schema

The logical schema below lists each table with its key attributes, primary/foreign keys, and a brief normalization note. All tables are normalized to Third Normal Form (3NF); the DOCUMENT_CHUNK and EMBEDDING tables are split out specifically to avoid storing large vector payloads redundantly with document metadata.

Table	Key Columns	Notes
USER	user_id (PK)	name, email (unique), role, created_at
CONVERSATION	conversation_id (PK), user_id (FK)	started_at, status (open/closed)
MESSAGE	message_id (PK), conversation_id (FK)	sender_type (customer/bot/agent), content, timestamp
TICKET	ticket_id (PK), conversation_id (FK), agent_id (FK)	priority, status (created on escalation)
AGENT	agent_id (PK)	name, team
KNOWLEDGE_DOCUMENT	doc_id (PK)	title, source_type (FAQ/manual/ticket), updated_at
DOCUMENT_CHUNK	chunk_id (PK), doc_id (FK), embedding_id (FK)	chunk_text, chunk_index
EMBEDDING	embedding_id (PK)	vector (float array), model_version

FEEDBACK	feedback_id (PK), message_id (FK)	rating (1–5), comment
MODEL_VERSION	model_version_id (PK)	name, trained_at, mlflow_run_id

4.6 Data Flow & System Behavior

4.6.1 Data Flow Diagram — Context Level (Level 0)

The context-level DFD treats the entire chatbot system as a single process and shows the data exchanged with each external entity: customers, support agents, administrators, and the external LLM provider.

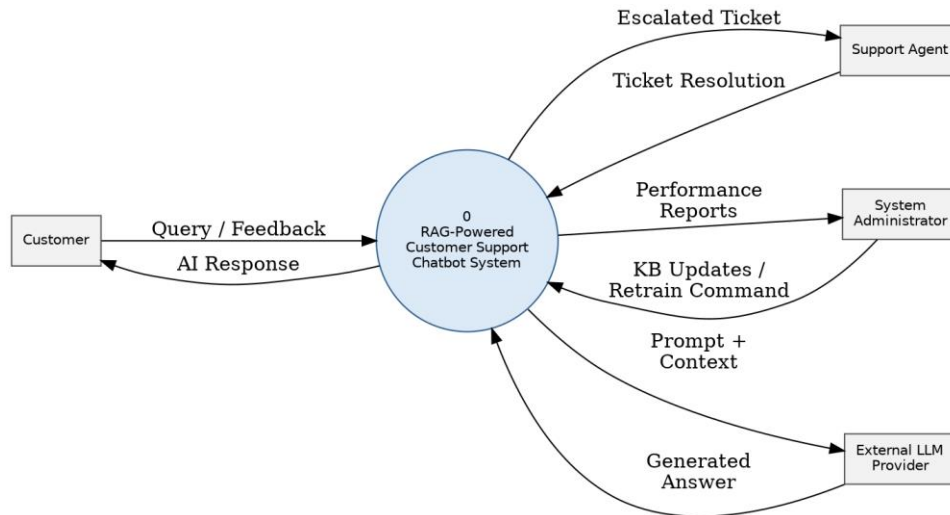


Figure 4.4 — Context-Level Data Flow Diagram

4.6.2 Data Flow Diagram — Level 1 (Detailed)

The Level 1 DFD decomposes the system into five core processes — Query Preprocessing, Document Retrieval, Response Generation, Feedback & Monitoring, and Knowledge Base Management — and shows how they interact with four persistent data stores: the vector store, the conversation/ticket database, the knowledge base, and the feedback/metrics store.

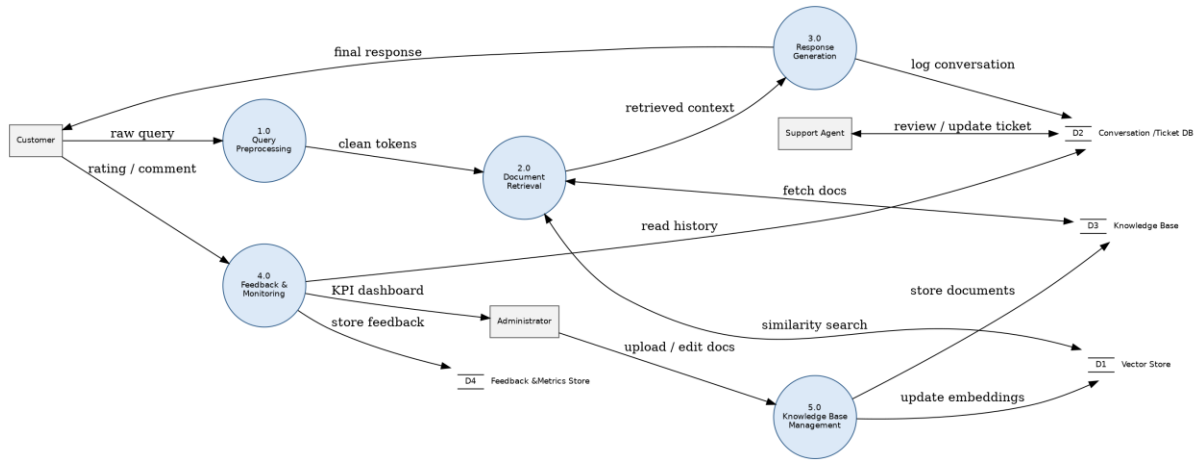


Figure 4.5 — Level 1 Data Flow Diagram

4.6.3 Sequence Diagram

The sequence diagram illustrates the runtime interaction between components when a customer submits a query: the API Gateway forwards the request to the RAG Pipeline, which preprocesses the query, retrieves relevant context from the Vector Store through the Retriever, calls the Generator (LLM) to produce an answer, post-processes the result, logs metrics to MLflow, and returns the final response — with an optional feedback step afterward.

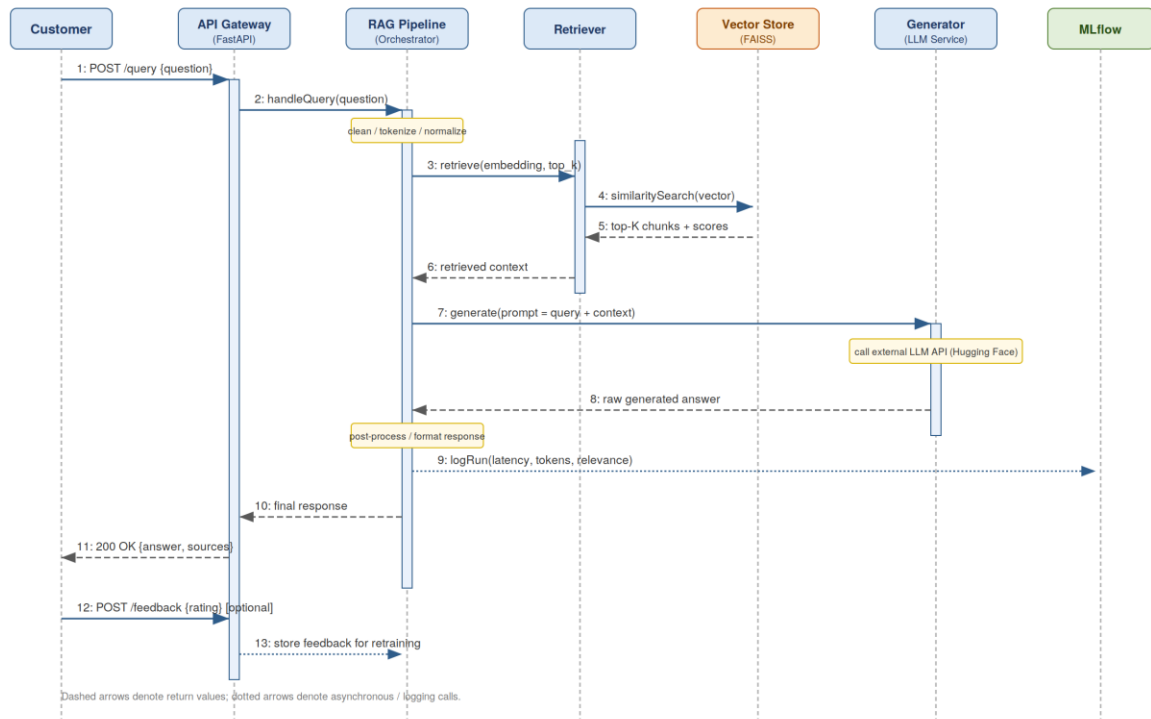


Figure 4.6 — Sequence Diagram: Customer Query → RAG Response

4.6.4 Activity Diagram

The activity diagram captures the end-to-end workflow of handling a customer query, including the decision point that determines whether the system responds automatically or escalates to a human agent, and the subsequent logging and feedback steps.

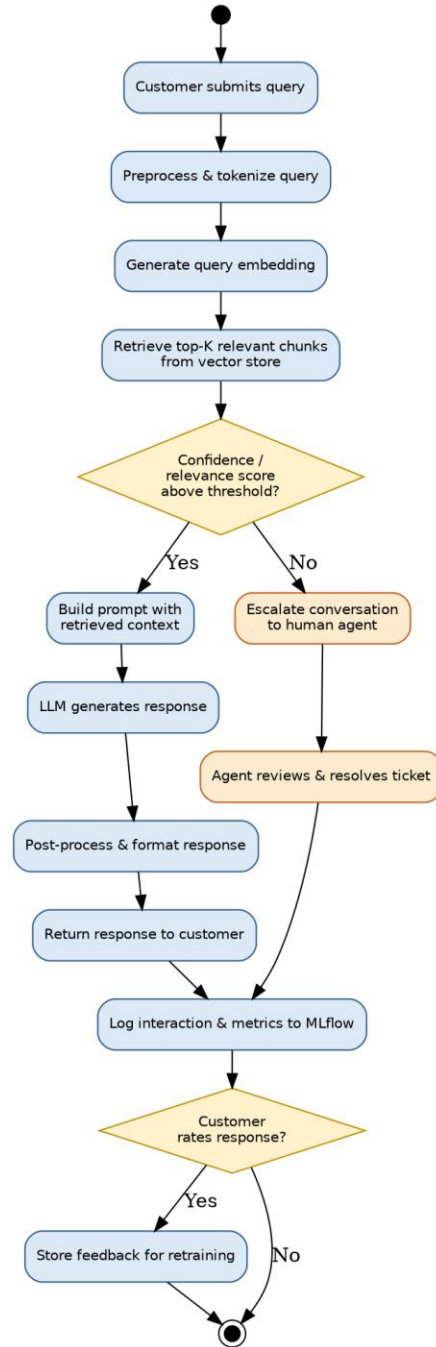


Figure 4.7 — Activity Diagram: Query Handling Workflow

4.6.5 State Diagram

The state diagram models the lifecycle of a customer conversation / support ticket as it moves from creation through automated bot handling, optional escalation and agent review, resolution, closure, and possible reopening.



Figure 4.8 — State Diagram: Conversation / Ticket Lifecycle

4.6.6 Class Diagram

The class diagram defines the core object model of the system: domain classes such as User, Conversation, Message, Ticket, and Agent; RAG-pipeline classes such as RAGPipeline, Retriever, Generator, and VectorStore; and supporting services for feedback and monitoring. Associations reflect the relationships already established in the ER diagram, while the RAGPipeline composition relationships reflect the internal software design.

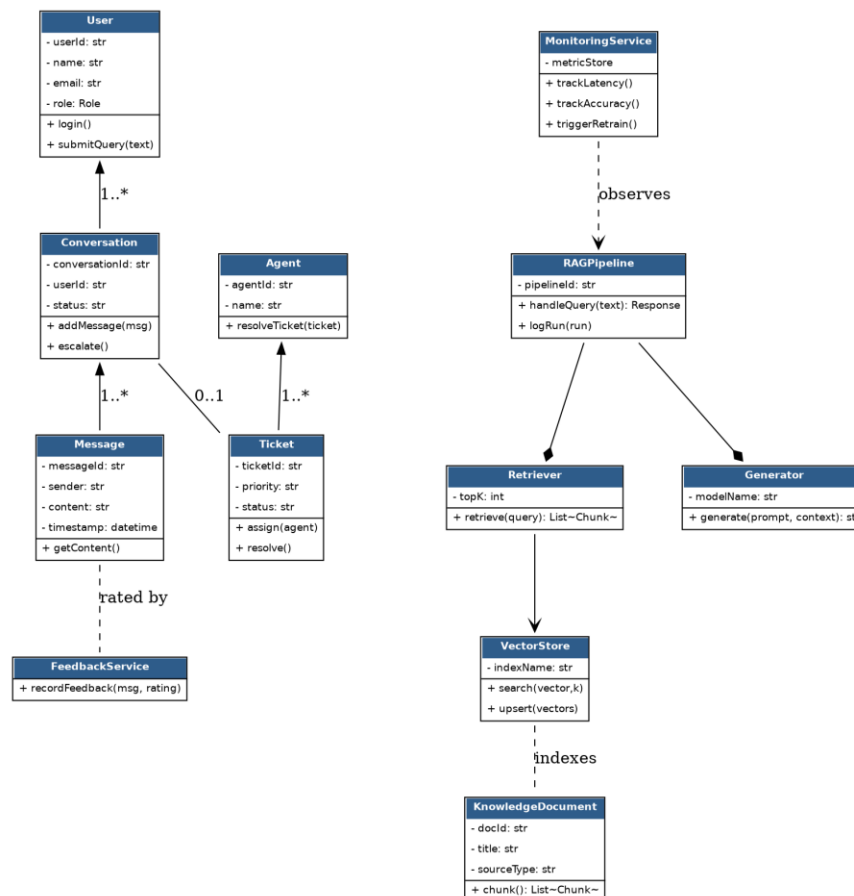


Figure 4.9 — Class Diagram

5. UI/UX Design & Prototyping

This section presents the interface design work for the Customer Support RAG-Powered Intelligent Chatbot: wireframes and mockups for the three primary user-facing screens, and the design system (visual guidelines) that governs color, typography, and accessibility across the product.

5.1 Wireframes & Mockups

Three core screens were designed to support the system's three primary user roles — the customer, the support agent, and the administrator. Each wireframe below shows the intended layout, key components, and interaction points at a structural (low fidelity) level; visual polish and final styling are applied in high-fidelity mockups during implementation.

5.1.1 Customer Chat Widget

The chat widget is embedded on the company's website and mobile app. It shows a conversational thread with distinct styling for bot and customer messages, inline source citations for transparency, quick-reply chips for common actions (helpful / not helpful / talk to agent), and a persistent input bar.

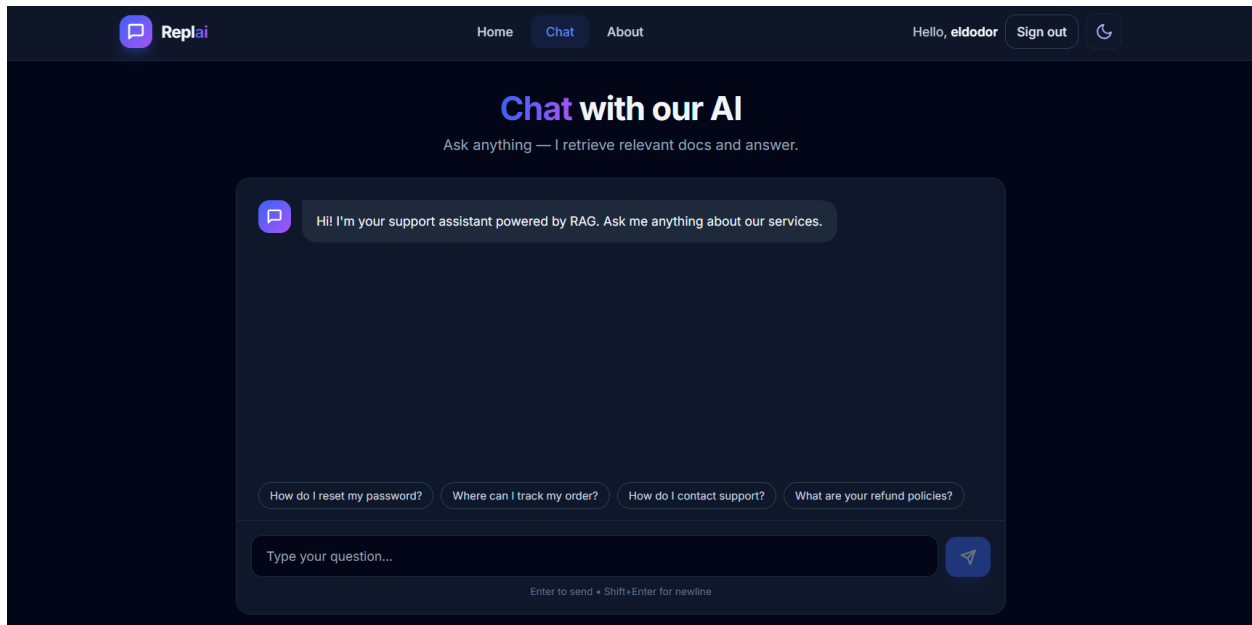


Figure 5.1 — Wireframe: Customer Chat Widget

- Header: assistant name/avatar and live status indicator.
- Message thread: bot responses (left-aligned) vs. customer messages (right-aligned), with a source citation card beneath grounded answers.
- Quick-reply to chips: one-tap feedback and escalation trigger.
- Input bar: text field with send button; supports file/screenshot attachment (not shown).

5.2 UI/UX Guidelines

The interface design follows a consistent set of principles: clarity over decoration, fast recognizable feedback, and trust through transparency. Because the system generates answers automatically, every AI response is visually distinguished from human input and, where possible, backed by a visible source citation so customers and agents can verify the information rather than blindly trust it.

- Consistency: shared color, spacing, and component patterns across the customer widget, agent console, and admin dashboard so users transfer knowledge between screens.
- Transparency: AI-generated responses are visually marked and paired with source citations; confidence-driven escalation is surfaced to the user rather than hidden.
- Progressive disclosure: agents and admins see dense data (metrics, retrieved context) only in the panels where it is actionable, keeping the customer-facing widget lightweight.
- Responsiveness: layouts adapt from a full agent console on desktop to a single-column chat view on mobile.

The design system below defines the color palette, typography, and accessibility rules applied consistently across all screens.

Color Palette



Typography

Aa Headings — Segoe UI / Calibri Bold
H1 24pt · H2 18pt · H3 14pt — Navy #1F3864

Aa Body Text — Calibri / Segoe UI Regular
11pt body, 1.4 line-height — Charcoal #333333 on white

Aa Captions / Meta — Calibri Italic
9–10pt, Grey #8C97A1, used for timestamps and source citations

Accessibility Notes

Text/background contrast ratio $\geq 4.5:1$ (WCAG 2.1 AA) · All interactive elements keyboard-navigable and screen-reader labeled
Minimum touch target 44x44px · Status conveyed by icon + text, never color alone · Focus states clearly visible

Figure 5.4 — UI/UX Design Guidelines: Color Palette & Typography

Accessibility is treated as a first-class requirement rather than an afterthought: all text meets a minimum 4.5:1 contrast ratio against its background (WCAG 2.1 AA), interactive elements are fully keyboard-navigable and labeled for screen readers, touch targets are at least 44x44px, and status is always conveyed through an icon-plus-text combination rather than color alone, so the interface remains usable for people with color-vision deficiencies.

6. System Deployment & Integration

This section describes how the Customer Support RAG-Powered Intelligent Chatbot is built and deployed in practice: the concrete technology stack behind each layer of the architecture, the physical/cloud infrastructure the software runs on, and the high-level software components and their dependencies.

6.1 Technology Stack

The technology choices below map directly onto the layered architecture introduced in Section 4.4, favoring managed Azure services to minimize operational overhead while keeping the RAG components portable and independently testable.

Layer	Technology	Purpose
Frontend	React / JavaScript (chat widget), Responsive Web (agent & admin consoles)	Customer chat widget, support agent console, and administrator dashboard UIs.
Backend / API	Python, FastAPI	REST gateway, authentication, request routing, and orchestration of the RAG pipeline.
RAG / NLP	Hugging Face Transformers, LangChain-style orchestration	Query preprocessing, embedding generation, retrieval orchestration, and response generation.
Vector Database	FAISS / Azure Cognitive Search	Storage and similarity search over document embeddings.
Operational Database	Azure SQL Database / Azure Cosmos DB	Users, conversations, messages, tickets, and feedback records.
Document Storage	Azure Blob Storage	Raw knowledge base documents, manuals, and FAQ sources.
MLOps	MLflow, Azure Machine Learning	Experiment tracking, model/embedding versioning, and automated retraining pipelines.
Authentication & Security	Azure Active Directory, API Keys, TLS 1.2+	Identity, access control, and encryption in transit.
Deployment / Hosting	Azure App Service, Azure Kubernetes Service (AKS), Docker	Hosting the API gateway and containerized RAG services.
Monitoring	Azure Monitor, MLflow dashboards	Latency, drift, and accuracy monitoring; KPI reporting for administrators.

6.2 Deployment Diagram

The deployment diagram maps the software components onto physical and cloud infrastructure nodes. Client devices communicate with an Azure App Service instance hosting the API gateway, which delegates RAG processing to a set of containerized services running on Azure Kubernetes Service. These services call out to the vector index, the operational database, blob storage for knowledge documents, and an external LLM provider, while continuously logging to an Azure Machine Learning workspace that hosts the MLflow tracking server and the automated retraining pipeline.

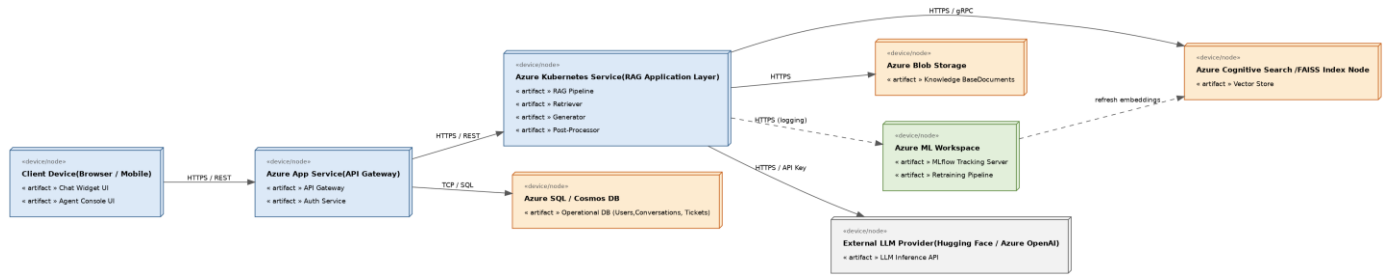


Figure 6.1 — Deployment Diagram

6.3 Component Diagram

The component diagram shows the system decomposed into high-level, independently deployable software components and the dependencies between them, using UML «use» dependency arrows. The Chat Widget and Agent Console UIs depend only on the API Gateway; the Gateway delegates authentication, ticketing, and RAG processing to dedicated components; and the RAG Pipeline composes the Retriever and Generator components, which in turn depend on the Vector Store component and the external LLM API respectively.

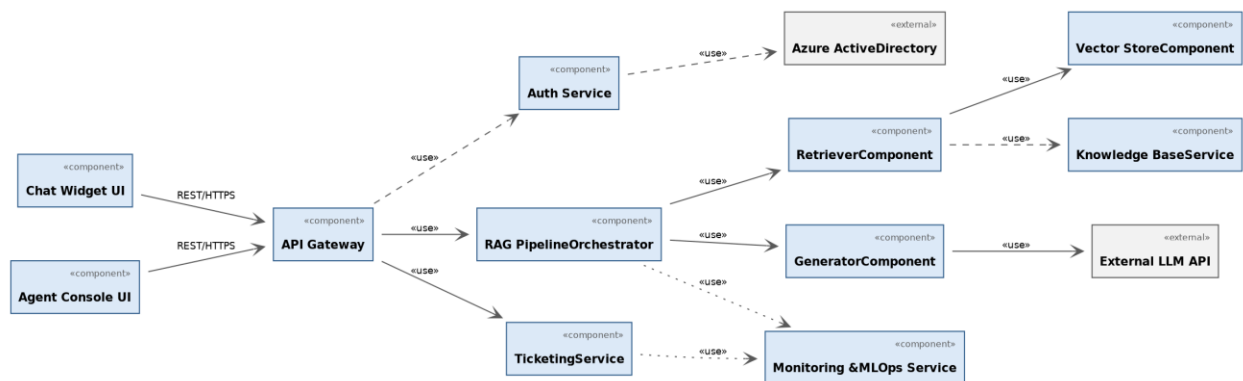


Figure 6.2 — Component Diagram